

Continuous Internal Evaluation- CIE - II conducted at the end of 8th week		
Sl No	Assessment of parameter	Marks
1	Capstone project Details: • Description of Technology Used • Details of Hardware devices • Details of software products • Programming languages • Descriptions of the components in the system • Component diagrams and required design if any • Construction or Fabrication details • Any other information needed to execute the capstone project	80
	Total	80

ANSWERS TO CIE-II ASSESSMENT PARAMETERS

1. Description of Technology Used

K.A.R.M.A is a cyber threat deception and Security Information and Event Management (SIEM) platform designed to lure, detect, classify, and neutralize cyber adversaries in real time. The key technologies used are:

- **1. Cyber Deception (HoneyPot Technology):** Simulates genuine vulnerable services (SSH, Web, FTP, Telnet, RDP) to trap unauthorized reconnaissance and exploits safely in memory.
- **2. Asynchronous Event-Driven Architecture:** Built using Python FastAPI and Uvicorn ASGI server, handling hundreds of concurrent connection probes asynchronously without blocking.
- **3. Real-Time WebSocket Streaming:** Full-duplex WebSocket channels stream incoming attack events instantly to the web dashboard with sub-45ms latency.
- **4. MITRE ATT&CK Threat Classification:** Automatically maps observed attacker payloads and commands to official MITRE Enterprise TTP identifiers (T1110, T1059, T1046, T1190, T1078).
- **5. Artificial Intelligence (DeepSeek AI):** Powered by the DeepSeek-V3 neural engine (`deepseek-chat`). Ingests live database statistics to generate incident triage reports and firewall rules.
- **6. Interactive Geospatial Mapping:** Integrated Leaflet.js with free ESRI Dark Canvas and OpenStreetMap tile layers, resolving attacker coordinates via GeoIP.
- **7. Zero-Storage Cryptographic Toolkit:** Browser Web Crypto API calculates SHA-256, SHA-512, and MD5 hashes in client memory with zero server transmission, guaranteeing 100% user privacy.

2. Details of Hardware Devices

K.A.R.M.A runs on standard commodity computer hardware without needing expensive server racks. The physical hardware specifications used are:

- **1. Host Processor (CPU):** Intel Core i5-1135G7 (4 Cores, 8 Threads @ 2.40 GHz up to 4.20 GHz). Handles FastAPI server, socket daemons, and database queries.
- **2. System Memory (RAM):** 16 GB DDR4 @ 3200 MHz. Maintains in-memory WebSocket queues, GeoIP cache, and virtual terminal shell buffers.
- **3. Primary Storage:** 512 GB NVMe M.2 SSD. Provides high read/write speeds for SQLite database transactions and CSV audit logs.
- **4. Network Interface Cards:** Gigabit Ethernet (10/100/1000 Mbps RJ45) + Intel Wi-Fi 6 AX201 (802.11ax). Binds decoy listener ports on local LAN IP (e.g. 10.30.129.46).
- **5. Attacker Simulation Machine:** Secondary laptop (Intel Core i3, 8GB RAM, Kali Linux / Windows). Used to run Nmap port sweeps, Hydra password attacks, and curl exploit tests.
- **6. Hardware Safety Isolation:** All decoy services run in user space in Python memory (`shell=False`). Physical host hardware, SSD controllers, and operating system kernels remain completely safe.

3. Details of Software Products

The software stack consists of standard open-source products, database engines, and AI APIs:

- **1. Operating Systems:** Windows 10/11 Pro 64-bit (Host development platform) and Ubuntu 22.04 LTS (Emulated guest system).
- **2. Programming Runtime:** Python 3.10+ standard distribution with asyncio, socket, and threading libraries.

- **3. Backend Web Server:** FastAPI v0.110 (Async web framework) and Uvicorn v0.28 (High-speed ASGI web server).
- **4. SSH Library:** Paramiko v3.4 (OpenSSH protocol emulation with persistent 2048-bit RSA host keys).
- **5. Database Engine:** SQLite3 with Write-Ahead Logging (WAL Mode) for high-speed concurrent database writes.
- **6. AI Neural Engine:** DeepSeek AI REST API (`deepseek-chat`) for live threat reasoning and .EML email phishing forensics.
- **7. Frontend UI Libraries:** Leaflet.js v1.9.4 (World Map) and Chart.js v4.4 (Live event volume and doughnut charts).
- **8. Desktop GUI & Packaging:** Python Tkinter/Ttk (Control panel) and PyInstaller v6.4 (Standalone .exe packager).

4. Programming Languages

The system is implemented using four core programming languages:

- **1. Python 3.10+:** Used for core backend logic, multi-sensor socket daemons, Paramiko SSH server, SQLite database queries, MITRE regex classification, threat scoring formula, DeepSeek AI API integration, and the Tkinter desktop launcher.
- **2. JavaScript (ES6+):** Used for single-page dashboard interactivity (app.js), consuming real-time WebSocket telemetry, rendering Leaflet.js world map coordinates, animating Chart.js event graphs, and client-side Web Crypto API hashing.
- **3. HTML5:** Used for semantic layout structure, modal dialogs, navigation tabs, KPI metric cards, and drag-and-drop email file upload zones.
- **4. CSS3:** Used for the custom design system tokens (HSL colors), Glassmorphism effects (backdrop-filter: blur), responsive CSS Grid layouts, and the Cyber Dark / Crisp Light theme engine.
- **5. SQL (SQLite):** Used for relational database tables (`events`, `attacker_profiles`, `quarantine`), index creation, and fast parameterized queries.

5. Descriptions of the Components in the System

The platform consists of nine modular subsystems:

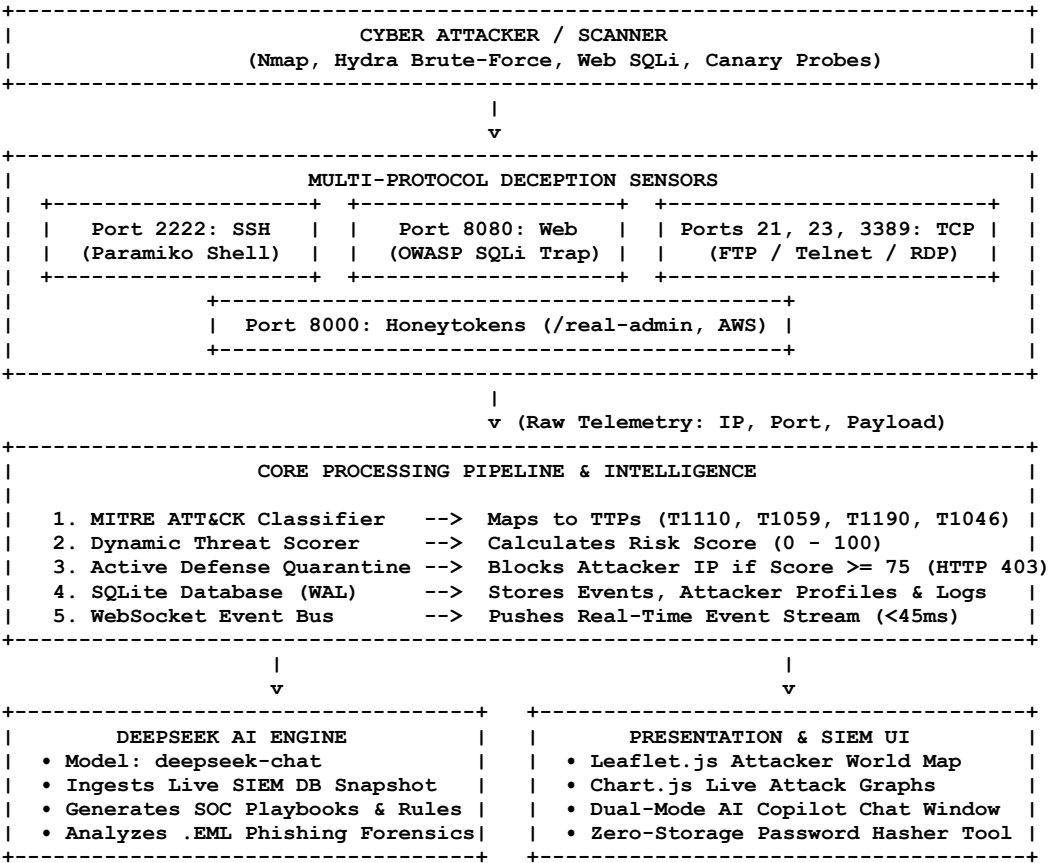
- **1. OpenSSH Honeygot (Port 2222):** Virtual SSH server on Port 2222 with RSA host keys. Simulates Linux bash login, traps brute-force passwords (MITRE T1110), and logs interactive commands (`whoami`, `cat /etc/passwd`) safely without host execution (MITRE T1059).
- **2. Web Admin Decoy (Port 8080):** Simulated corporate web login portal on Port 8080. Evaluates input requests against regex patterns to trap SQL Injection, XSS, and Path Traversal attacks (MITRE T1190).
- **3. Raw TCP Decoy Listeners:** Asynchronous raw TCP daemon sockets on Ports 21 (FTP), 23 (Telnet), and 3389 (RDP). Traps Nmap port scans and banner grabs (MITRE T1046).
- **4. Production Honeytoken Vault:** Decoy AWS API keys (`AKIA...`) and hidden canary routes (`/real-admin`, `/api/v1/auth/keys`). Any access attempt immediately triggers a critical alert (Threat Score: 100/100, MITRE T1078).
- **5. MITRE ATT&CK Classifier:** Rule-based signature engine that maps captured raw attack payloads directly to official MITRE Enterprise technique IDs.
- **6. Dynamic Threat Scorer:** Calculates threat scores (0–100) based on sensor weight, payload maliciousness, and attack frequency. Categories: Low (<30), Medium (30–74), Critical (>=75).

- **7. Active Defense Quarantine:** Automatically isolates adversary IPs with threat scores ≥ 75 , blocking further access with a custom HTTP 403 deception banner.
- **8. KARMA AI SOC Copilot:** AI assistant using DeepSeek API. Ingests live database statistics to generate incident reports and firewall rules. Also analyzes uploaded `.eml` phishing email headers.
- **9. Cloud SIEM Dashboard:** Responsive single-page web dashboard with Leaflet world map, Chart.js trends, live event tables, and Tkinter desktop launcher GUI.

6. Component Diagrams and Required Design

Below is the simple block diagram representing the complete system architecture, designed to be easily drawn in a notebook using a pen and ruler:

K.A.R.M.A System Architecture Block Diagram (Hand-Drawable Schema):



How to draw this in your notebook:

- **Step 1:** Draw a top box for 'Attacker' with downward arrows.
- **Step 2:** Draw a wide box for 'Deception Sensors' containing 4 sub-boxes: SSH (2222), Web (8080), Raw TCP (21/23/3389), and Honeytokens (8000).
- **Step 3:** Draw a middle box for 'Core Engine' listing MITRE Classifier, Threat Scorer (0-100), Active Quarantine, SQLite Database, and WebSocket Bus.
- **Step 4:** Draw two bottom boxes: 'DeepSeek AI Engine' on the left and 'Web SIEM Dashboard' on the right, connected by downward arrows.

7. Construction or Fabrication Details

The software was constructed through modular steps:

- **1. SSH Honeypot:** Created ``backend/sensors/ssh_sensor.py`` subclassing ``paramiko.ServerInterface``. Generated a 2048-bit RSA key (``ssh_host_rsa_key``) to keep host fingerprint constant. Traps passwords and logs shell commands in memory.
- **2. Web Honeypot:** Created ``backend/sensors/web_sensor.py`` on Port 8080. Added regex scanners checking for SQL injection (``' OR '1'='1``), XSS (``<script>``), and directory traversal (``../etc/passwd``).
- **3. Raw TCP Sockets:** Created daemon threads in ``backend/sensors/raw_tcp_sensor.py`` binding raw sockets on Ports 21, 23, and 3389. Sockets accept handshakes, log IP/port, and send fake banners.
- **4. Honeytoken Vault:** Created ``backend/honeytokens.py`` seeding routes ``/real-admin`` and ``/api/v1/auth/keys``. Any HTTP request triggers immediate quarantine.
- **5. WebSocket Manager:** Built ``backend/engine/websocket_manager.py`` using Python ``asyncio`` to broadcast newly logged attack events to browser clients in real time.
- **6. DeepSeek AI Copilot:** Created ``backend/engine/ai_chat.py`` communicating with DeepSeek API. Pulls live database metrics and inserts them into system prompts for accurate incident response.
- **7. GUI Launcher:** Created ``launcher_gui.py`` using Python Tkinter for one-click startup and compiled standalone package using PyInstaller in ``setup.py``.

8. Any Other Information Needed to Execute the Capstone Project

Essential setup, startup steps, and test demonstration commands:

A. System Prerequisites:

- Operating System: Windows 10/11 or Linux Ubuntu 20.04+ (64-bit).
- Python Environment: Python 3.10+ installed with pip.
- Web Browser: Google Chrome, Microsoft Edge, or Mozilla Firefox.

B. Step-by-Step Execution Commands:

- Step 1 (Install Dependencies): ``pip install -r requirements.txt``
- Step 2 (Set AI Key): ``set DEEPSEEK_API_KEY=sk-5cbb5986a1c145cba084ac78a616eb01``
- Step 3 (Run Application): ``python run.py`` (Launches desktop GUI and starts all sensors)
- Step 4 (Access SIEM Web Dashboard): Open browser to ``http://localhost:8000/``

C. Verification Attack Commands for Viva Demonstration:

- Test SSH Trap: ``ssh admin@localhost -p 2222`` -> enter password -> run ``whoami``, ``cat /etc/passwd``
- Test Web Trap: Open browser to ``http://localhost:8080/`` -> input username ``' OR '1'='1``
- Test Port Scan: ``nmap -sS -p 21,23,8080,2222,3389 localhost``
- Test Honeytoken: Open browser to ``http://localhost:8000/api/v1/auth/keys`` -> observe instant IP quarantine